

Proyecto Práctico Módulo 7 Antonio Ruiz Guirao

Chef Rapidin

Modelo usado

<https://huggingface.co/Qwen/Qwen2.5-3B>

Índice

1. Introducción	2
1.1 Objetivo del trabajo	2
1.2 Motivación	2
2. Estructura del Proyecto	2
3. Tecnologías Utilizadas 	3
4. Metodología Empleada	3
5. Funcionamiento del Sistema	4
6. Modelos Utilizados	6
Modelo generativo	6
Ingeniería de Prompt	6
Modelo de embeddings	6
Recuperación de Información (RAG)	7
7. Dataset Empleado	7
8. Resultados Obtenidos	7
9. Problemas Encontrados	9
10. Posibles Mejoras Futuras	9
11. Instalación y Ejecución	9
12. Conclusiones	10

1. Introducción

1.1 Objetivo del trabajo

El objetivo del proyecto buscaba crear un aplicación para obtener recetas con los ingredientes disponibles de forma rápida.

Chef Rapidín es una app web, desarrollada con Streamlit, capaz de generar recetas únicas, a partir de los ingredientes disponibles del usuario.

Combinando un modelo de lenguaje, con técnicas de RAG (Generación Aumentada por Recuperación), permite crear recetas sacando información de una base de datos, y crear nuevas a partir de estas.

Junto con unos filtros, para tipos de dietas, alergias y momentos del día.

1.2 Motivación

La falta de planificación en la cocina, lleva a repetir los mismos 3 platos, tirando nutrientes por la borda y aburriendo al paladar y la vista. Se busca dar recetas rápidas de crear y con lo disponible para cada uno.

Usando a la IA se agiliza y optimiza un proceso en la vida real, aunque quizás quite creatividad a la hora de inventar platos, pero da sugerencias pudiendo ampliar las opciones del usuario y dejando la parte divertida de comerlo y ampliar sus recetas con ingredientes que posee.

2. Estructura del Proyecto

En la Imagen 1, se aprecia la estructura del proyecto, teniendo los archivos siguientes:

- **chef.py** → Toda la logica y operaciones del chef, junto a la app web.
- **historial_cocina.json** → Historial de cocina para no repetir platos
- **requirements.txt** → dependencias necesarias para ejecutar el proyecto.

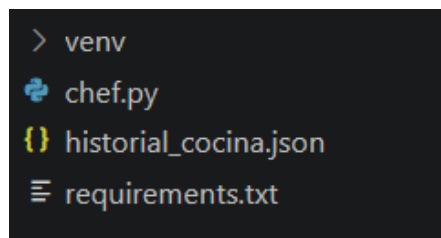


Imagen 1: Estructura de archivos del proyectos

Los componentes principales del proyecto serían:

- Interfaz gráfica web con Streamlit.
- Sistema de recuperación RAG.
- Motor de generación de recetas.
- Sistema de historial.
- Filtrado dietético y de alergias.

3. Tecnologías Utilizadas

Tecnología	Función
Python	Lenguaje principal
Streamlit	Interfaz web
PyTorch	Ejecución de Modelos
Transformers	Carga del LLM
SentenceTransformers	Embeddings
Scikit-Learn	Similitud coseno
Pandas	Procesamiento de datos
NumPy	Operaciones numéricas
HuggingFace Datasets	Dataset de recetas
BitsAndBytes	Cuantización 4 bits

4. Metodología Empleada

Para el desarrollo se siguió una metodología modular:

1. **Entrada del usuario**

El usuario introduce:

- Ingredientes disponibles.
- Restricciones alimenticias (alergias)
- Tipo de dieta (vegano, vegetariano, dieta)
- Momento del día (cena, comida)

2. **Filtrado de recetas**

Se filtra el dataset, filtrando solo los datos necesarios y eliminando filas y columnas innecesarias

3. **Recuperación semántica**

Recetas se convierten a embeddings, y luego se calcula el coseno entre los ingredientes del usuario, y los embeddings de las recetas

4. **Generación**

Gwen recibe los ingredientes, contexto y restricciones, y genera 2 recetas nuevas en formato JSON válido, para mostrarlas luego al usuario.

5. Funcionamiento del Sistema

El diagrama de la Imagen 2 muestra el ciclo de vida de una consulta en Chef Rapidín V2. La clave del sistema es que no enviamos los ingredientes del usuario a ciegas, primero pasan por un filtro estricto de seguridad alimentaria y una búsqueda vectorial con el RAG. De esta forma el modelo puede generar recetas con sentido y creativas.

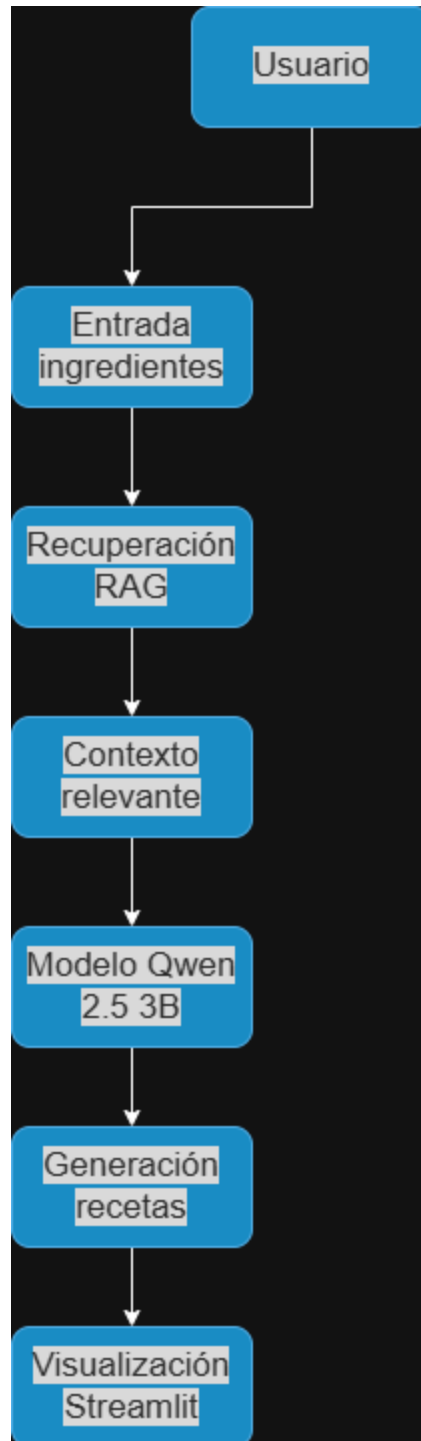


Imagen 2. Diagrama de funcionamiento del Sistema

6. Modelos Utilizados

Modelo generativo

Al tener un GPU, una RTX 5070 Ti, se dispone de suficiente potencia de cálculo para correr un módulo pesado, por eso empleó Qwen.

- **Qwen 2.5 3B Instruct**
- Aproximadamente 3 billones de parámetros y optimizado para conversaciones.
- Optimizado mediante cuantización de 4 bits.

Ingeniería de Prompt

El modelo generativo recibe un mensaje de sistema para que responda de una manera concreta. El prompt se diseñó para:

- Garantizar respuestas estructuradas.
- Evitar errores de parseo.
- Respetar alergias y dietas.
- Aprovechar el contexto recuperado por RAG.
- Generar recetas variadas y creativas.
- Dar personalidad al Chef.
- Establecer reglas para responder.

El prompt se puede ver en la siguiente imagen

```
system prompt = """
Eres un Chef experto. Usa el contexto de recetas reales proporcionado para crear recetas creativas.

Recetas de referencia (USA ESTAS COMO BASE):
{contexto_rag}

Reglas estrictas:
- Si las recetas de referencia usan ingredientes que tengo, úsalas.
- REGLA DE SALUD CRÍTICA: No utilices bajo ningún concepto ingredientes prohibidos por el usuario. Alergias a evitar: {alergias}. Dieta obligatoria: {'', '.join(tipo_dieta)}.
- Respuesta SOLO en JSON. Sin texto adicional ni bloques de código markdown extraños.
- Humor sarcástico e irónico fuerte.
- Asegúrate de que las acciones culinarias en español tengan sentido (usa verbos como picar, trocear, batir, nunca inventos raros).
- Formato JSON con claves: opcion_1, opcion_2, opcion_3. Cada una con nombre, tecnica, tiempo, pasos (lista).
"""
```

Imagen 3: Prompt de instrucciones de Qwen

Modelo de embeddings

- **sentence-transformers/all-MiniLM-L6-v2**: para representar las recetas con sentido semántico.

Recuperación de Información (RAG)

Se empleó un RAG para mejorar las respuestas del modelo y el proceso es el siguiente:

1. Se codifican las recetas.
2. Se calcula similitud coseno.
3. Se recuperan las recetas más relevantes según el usuario.
4. Se utilizan como contexto para la generación.

7. Dataset Empleado

Se empleó el dataset de [recipes-with-nutrition](#) disponible en Hugging face, del cual se utilizaron 8000 filas para reducir su tamaño, y las columnas fueron:

- recipe_name
- ingredients
- ingredient_lines
- health_labels
- meal_type
- dish_type

8. Resultados Obtenidos

Al final el sistema es capaz de:

- **Generar recetas personalizadas y coherentes, con un toque especial.**
- **Adaptarse a dietas específicas.**
- **Excluir ingredientes por alergias.**
- **Recuperar recetas similares mediante RAG.**
- **Mantener un historial de recetas guardadas**

Ejemplo de resultado del JSON de salida.

```
[
  {
    "fecha": "2026-06-05",
    "plato": "Arroz con Jamón y Tomate",
    "ingredientes": "jamón serrano, queso, arroz, huevo, pimienta, tomate ",
    "tecnica": "Arroz a la plancha"
  }
]
```

En la Imagen 4 se puede observar un ejemplo de ejecución y el resultado en la interfaz mediante Streamlit.

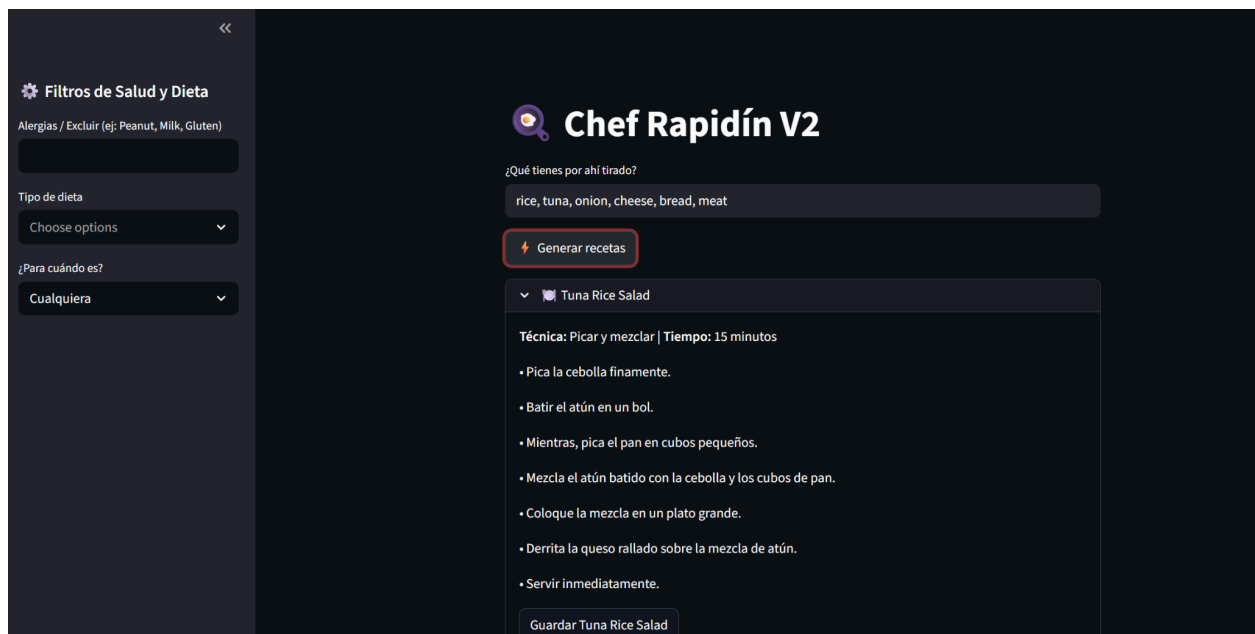


Imagen 4: Interfaz web del chef y ejemplo resultado

9. Problemas Encontrados

- Uno de los problemas fue el consumo de memoria, con otros modelos, así que se usó Qwen mas bajo, y se cuantizo usando “BitsAndBytes” en 4 bits.
- Errores con el formato JSON, y saltos de las restricciones en el prompt, ignorando el prompt y generando valores incorrectos. Para ello se implementó una limpieza y validación del archivo JSON, junto a un filtrado previo de ingredientes por alergia o dietas.

10. Posibles Mejoras Futuras

El proyecto podrían incorporarse varias mejoras, como las siguientes:

- Incorporar información nutricional de los platos.
- Mostrar macros del plato.
- Añadir generación de imágenes de los platos.
- Sistema de valoración de recetas.
- Perfil personalizado del Usuario, guardando configuración.
- Uso de bases vectoriales.

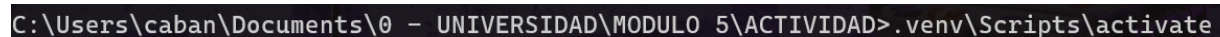
11. Instalación y Ejecución

1. **Primero crear el entorno virtual** (si no está creado):

- `python -m venv .venv`

2. **Activar el entorno virtual:**

- `.venv\Scripts\activate`
- Como se ve en la imagen 5



```
C:\Users\caban\Documents\0 - UNIVERSIDAD\MODULO 5\ACTIVIDAD>.venv\Scripts\activate
```

Imagen 5: Activar el entorno

3. Una vez activado saldrá (.venv) al inicio del terminal, **entonces instalar todas las dependencias:**

- `pip install -r requirements.txt`
- En la imagen 6 se ve el entorno activado, y como se instalan las dependencias

```
(.venv) C:\Users\caban\Documents\0 - UNIVERSIDAD\MODULO 5\ACTIVIDAD>pip install -r requirements.txt
Collecting streamlit (from -r requirements.txt (line 1))
  Using cached streamlit-1.57.0-py3-none-any.whl.metadata (9.6 kB)
Collecting transformers (from -r requirements.txt (line 2))
  Downloading transformers-5.8.0-py3-none-any.whl.metadata (33 kB)
Collecting torch (from -r requirements.txt (line 3))
```

Imagen 6: Instalando dependencias

4. **Ejecutar la aplicación** una vez acabe, mediante:

- `streamlit run chef.py`

Se abrirá en el navegador predeterminado, la interfaz para poder preguntar al chef rapidin

12. Conclusiones

En conclusión, Chef rapidin, consigue combinar modelos generativos con técnicas de RAG construyendo un sistema inteligente, capaz de generar recetas personalizadas y creativas, adaptadas a las necesidades del usuario.

Junto con la recuperación semántica para mejorar la calidad de las respuestas, proporciona respuestas más coherentes y cocinables.